

BreachShield: A Privacy-Preserving OSINT Breach Intelligence Platform — Design, Verified Capabilities, and a Multi-Stage Audit of AI-Assisted Development Claims

Arvind D. H., Girishkumar N. M., Manoj, Premkumar Teli

Dept. of Information Science and Engineering, Acharya Institute of Technology, Bengaluru, India

Sushma T. M.

Project Guide, Assistant Professor, Dept. of Information Science and Engineering, Acharya Institute of Technology

Abstract—Breach-checking and dark-web exposure monitoring services face a structural privacy paradox: to learn whether an identifier has been compromised, users are typically asked to submit that identifier, in the clear, to a third-party server. This paper presents BreachShield, a privacy-preserving OSINT breach intelligence platform built across two academic project phases, and reports its capabilities as established through live, reproducible verification rather than as originally specified. The platform implements a client-side k-anonymity search protocol (5-hex-character SHA-256 prefix disclosure), a dual-channel out-of-band OTP identity gate, a pluggable OSINT source registry including a live Telegram scraper, a gradient-boosted breach-severity classifier (XGBoost, 89.2% test accuracy on 1,034 curated records), and a Merkle-tree-batched audit log anchored to a local EVM smart contract. Because substantial portions of the codebase were produced with AI coding-agent assistance, the development team adopted an explicit policy: no capability claim was retained in this paper without independent, live re-verification, carried out in three stages – dual independent audit, live cross-checking against running system behaviour, and independent third-party reproduction. This process both confirmed genuine results (a fully deterministic, checkpoint-loaded severity classifier; 101/101 backend and 50/50 frontend tests passing; a working live phishing-URL classifier) and surfaced fabricated or non-functional claims introduced during development (an unrelated, uncheckpointed comparison model set; a blockchain auto-deploy path that fails on a missing credential; and, separately, a set of latency figures and literature citations that were never measured or verified but had been written directly into paper-generation scripts). We report both the verified system and the audit methodology that distinguished it from what had merely been claimed.

Index Terms—breach intelligence, k-anonymity, OSINT, blockchain audit log, XGBoost, verification methodology, AI-assisted software engineering

I. INTRODUCTION

The continued growth of online services has been accompanied by a corresponding growth in credential leaks, phishing campaigns, and dark-web trade in compromised personal data [1]. Existing breach-notification tooling is largely reactive: users typically learn of compromise only after an incident is publicly disclosed, and the act of checking exposure itself often requires submitting a raw email address or phone number

to a third-party lookup service, converting a protective tool into a potential surveillance point.

This paper documents BreachShield, an OSINT-based breach intelligence platform developed across Project Phase 1 (conceptual design, exploratory literature survey, initial system requirements) and Project Phase 2 (production-oriented implementation) at the Department of Information Science and Engineering, Acharya Institute of Technology. The Phase 1 proposal described a broad cybersecurity suite spanning browser phishing protection, a general vulnerability scanner, third-party threat-feed integration, and blockchain-anchored logging. Phase 2 development concentrated engineering effort on a narrower, more defensible core: eliminating server-side query surveillance through client-side k-anonymity, and establishing a tamper-evident audit trail, while treating machine-learning severity scoring as a rigorously benchmarked but separately-tracked research artifact rather than an unverified production claim.

A second, methodological theme runs through this paper. A significant share of BreachShield’s implementation was produced with AI coding-agent assistance. During development, capability claims were at various points introduced into the codebase, its accompanying documentation, and early paper drafts that did not survive direct verification: an unrelated set of comparison models was, at one stage, believed to be loaded and functioning; a blockchain deployment step was reported as verified when it in fact failed; and a set of literature citations and system-latency figures were found, on inspection, to have been written directly into paper-generation tooling rather than measured or sourced. Rather than omit this from the record, we treat it as a relevant finding: this paper reports what a three-stage, live-verification-first methodology actually confirmed, separately from what had merely been asserted, and we regard the methodology itself as a contribution applicable to any team combining AI-assisted development with academic or peer-reviewed reporting.

The contributions of this paper are:

- 1) A privacy-preserving breach intelligence architecture combining client-side k-anonymity search, dual-channel

OTP identity gating, a pluggable multi-source OSINT registry, a benchmarked ML severity classifier, and a Merkle-batched blockchain audit log.

- 2) A three-stage verification methodology – dual independent audit, live cross-checking, and independent third-party reproduction – used to separate verified system behaviour from unverified or fabricated claims prior to publication.
- 3) A transparent, evidence-attached account of which sub-systems are verified and reproducible, which were found broken or fabricated and subsequently fixed, and which remain open, together with the specific root causes involved.

II. RELATED WORK

A. Phishing and Malicious-URL Detection

Character- and URL-level deep learning has been widely applied to phishing detection. Aljofey et al. [1] proposed a character-level CNN operating directly on raw URL strings, avoiding hand-engineered lexical features. Sahingoz et al. [2] benchmarked five deep architectures (ANN, CNN, RNN, BiRNN, attention networks) across roughly five million labelled URLs, reporting CNNs as the strongest performer for URL-only classification. These results motivate BreachShield’s use of a pretrained URL-transformer classifier (UrlBERT) on the live search path, rather than a hand-built lexical model.

B. Handling Class Imbalance in Security Classification

Severity and threat-classification datasets are frequently imbalanced toward common, lower-severity classes. Alsubaei et al. [3] report that applying SMOTE to a hybrid deep-learning phishing classifier improved accuracy from 83% to 98% on their benchmark, a result we return to in Section X when discussing BreachShield’s own CRITICAL-tier class-imbalance limitation.

C. Explainability and Few-Shot Detection

Pavani et al. [4] apply SHAP and LIME to malicious-URL classifiers to produce human-interpretable feature attributions, motivating our own use of per-model probability breakdowns rather than a single opaque severity score. Tawfik et al. [5] demonstrate few-shot BERT-based detection for emerging phishing campaigns with limited labelled data, a regime closely analogous to BreachShield’s 1,034-record breach-severity dataset.

D. OSINT and Dark-Web Threat Intelligence

Kühn et al. [6] provide a recent (2024) systematic evaluation of manual and semi-automated dark-web monitoring methodology, documenting both the intelligence value and the practical collection constraints of forum- and channel-based sources – constraints closely mirrored by BreachShield’s own migration from commercial threat-feed APIs (rate-limited, delayed disclosure) to a direct Telegram-channel scraper, discussed in Section III.

E. Tamper-Evident and Blockchain-Backed Audit Logging

Merkle’s original hash-tree construction [11] and Crosby and Wallach’s history-tree design [12] remain the foundational structures for append-only tamper-evident logging with efficient consistency proofs. Arabnouri et al. [7] recently proposed a checkpoint-based blockchain audit log that batches log entries by time interval to reduce on-chain storage – a design goal directly shared by BreachShield’s own Merkle-batching approach (Section VI), though our implementation anchors batches to a local EVM node rather than a public chain, a trade-off discussed in Section X.

F. Privacy-Preserving Compromise Checking

Sweeney’s k-anonymity model [8] and its application to health data by El Emam and Dankar [13] establish the general privacy guarantee BreachShield’s search protocol relies on. Li et al. [9] give the first rigorous security analysis of prefix-based compromised-credential-checking protocols of the kind popularised by HIBP Pwned Passwords, and propose stronger private-set-intersection-based alternatives; we discuss why BreachShield retains the simpler prefix-bucket design, and the resulting identity-linkage trade-off introduced by its OTP gate, in Section X.

G. Gradient-Boosted Classification and Tabular Data

Chen and Guestrin’s XGBoost [10] is the classifier underlying BreachShield’s severity triage. Its strong performance relative to deep architectures on BreachShield’s tabular breach-metadata is consistent with the broader empirical finding of Grinsztajn et al. [14], who show tree-based models remain state-of-the-art on medium-sized tabular datasets (their benchmark spans datasets in a similar order-of-magnitude range to BreachShield’s 1,034 records) due to neural networks’ bias toward overly smooth decision functions and sensitivity to uninformative features – both plausible explanations for the gap observed in Table III.

TABLE I
RELATED WORK SUMMARY AND RELATION TO BREACHSHIELD

Area	Relation to this work
URL/phishing DL [1], [2]	Motivates BreachShield’s pretrained URL-transformer classifier
Class imbalance [3]	Directly relevant to BreachShield’s CRITICAL-tier data scarcity (Sec. X)
Explainability [4], [5]	Motivates per-model probability reporting over a single opaque score
Dark-web OSINT [6]	Parallels BreachShield’s move to direct channel scraping
Blockchain audit logs [7], [11], [12]	Foundational structure and a recent, closely related batching design
k-anonymity / CCC	Direct theoretical basis [8], [13]; protocol-security analysis and stronger alternatives [9]
Tabular ML	Explains XGBoost’s margin over deep models [10], [14]

III. SYSTEM ARCHITECTURE

BreachShield is organised as a four-tier monorepo: a React 19 frontend, an Express 5 API gateway, a Python FastAPI OSINT microservice, and an Android Kotlin SMS relay gateway, illustrated in Fig. 1.

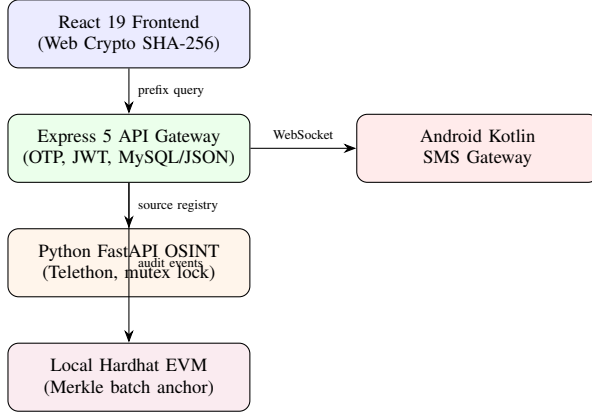


Fig. 1. Four-tier BreachShield architecture. Solid arrows denote live data paths verified by direct endpoint calls.

A. Frontend Web Dashboard

A React 19 single-page application performs client-side SHA-256 hashing via the W3C Web Crypto API, so that raw search targets never leave the browser in plaintext.

B. API Gateway

An Express 5 service manages dual-channel OTP issuance and verification, JWT session tokens, a pluggable breach-source registry, and MySQL 8 storage with an automatic JSON-file fallback layer.

C. OSINT Microservice

A Python FastAPI service integrates a Telethon-based Telegram scraper. Concurrent access to the underlying session file previously triggered `sqlite3.OperationalError: database is locked` under simultaneous asynchronous requests; this was resolved with an `asyncio.Lock` mutex (8.0 s timeout ceiling) serialising access to the Telegram client.

D. Android SMS Gateway

A companion Kotlin application maintains a persistent WebSocket connection to the backend, relaying OTP codes as carrier-compliant SMS in line with Indian telecom Distributed Ledger Technology (DLT) template requirements – a constraint that silently drops non-conforming payloads and required standardising the SMS template format during development.

IV. PRIVACY-PRESERVING SEARCH PROTOCOL

BreachShield’s search protocol follows the k-anonymity range-query construction popularised by HIBP Pwned Passwords [9] and grounded in Sweeney’s original definition [8]. The client normalises the target identifier T and computes $H = \text{SHA-256}(\text{normalize}(T))$, partitioning H into a 20-bit (5-hex-character) prefix P and a 236-bit suffix S . Only P is transmitted; the server returns the bucket $B(P) = \{(S_i, \text{Metadata}_i) : \text{SHA-256}(T_i)[0:5] = P\}$, and the client performs the final suffix match locally.

Algorithm 1 Client-Side k-Anonymity Range Query

Require: Target identifier T , session token τ

Ensure: Matched breach metadata R

```

1:  $T' \leftarrow \text{ToLower}(\text{Trim}(T))$ 
2:  $H \leftarrow \text{SHA-256}(T')$  ▷ computed in-browser
3:  $P \leftarrow H[0:5]$ ,  $S \leftarrow H[5:64]$ 
4:  $B \leftarrow \text{HTTP\_GET}(/api/search/range/P, \text{Bearer } \tau)$ 
5:  $R \leftarrow \emptyset$ 
6: for  $(S_i, M_i) \in B$  do
7:   if  $S_i = S$  then
8:      $R \leftarrow R \cup \{M_i\}$ 
9:   end if
10: end for
11: return  $R$ 
  
```

With $16^5 = 1,048,576$ prefix buckets, a corpus of N records yields an expected anonymity set $E[k] = N/1,048,576$ per prefix, and the information disclosed to the server is bounded by $I(T; P) \leq \log_2(16^5) = 20$ bits, independent of N .

A. Search-Gating Trade-off

Search endpoints are additionally gated behind OTP-verified JWT authentication (`verifyOtpToken` middleware) to prevent automated bucket-harvesting. This introduces a trade-off the k-anonymity guarantee alone does not resolve: the server authenticates the requester before serving a prefix bucket, so while the server never learns *which* target within a bucket the client is interested in, it does learn *who is making the query*. Li et al.’s analysis of compromised-credential-checking protocols [9] treats exactly this class of trade-off, and proposes private-set-intersection-based constructions that avoid it entirely at the cost of higher protocol complexity. BreachShield retains the simpler, OTP-gated design as a deliberate scope decision for a multi-tenant platform requiring accountable query logs; Section X discusses this explicitly rather than presenting the k-anonymity guarantee as unconditional.

V. MACHINE LEARNING SEVERITY TRIAGE

A. Dataset and Split

The severity classifier is trained on 1,034 curated breach records, partitioned with a stratified 70/15/15 split (train 723, validation 154, test 157; seed 42), summarised in Table II.

TABLE II
DATASET PARTITION AND CLASS DISTRIBUTION

Split	Total	LOW	MEDIUM	HIGH	CRITICAL
Train (70%)	723	49	341	291	42
Val (15%)	154	10	73	62	9
Test (15%)	157	11	74	63	9

B. Two Distinct Model Populations

BreachShield contains two architecturally similar but functionally distinct CNN/RNN/Transformer populations, a distinction established only through direct source inspection and is stated explicitly here to avoid the ambiguity that arose during this project’s own internal verification process:

- **Severity classifiers** (`severity_classifier.py`): CNN, RNN, and Transformer architectures that load real, trained checkpoints from disk via `torch.load/load_state_dict`, confirmed deterministic across repeated inference calls and across independent class instantiations. These feed the `severity_analysis` block of the live `/api/ai/analyze-threat` endpoint.
- **URL comparison models** (`cnn_model.py`, `rnn_model.py`, `transformer_model.py`): a separate set feeding the `/api/ai/compare` demonstration endpoint. These do not load any checkpoint and produce different output on every instantiation – verified directly, not merely assumed.

All ML results in this paper refer exclusively to the first, checkpoint-loaded population.

C. Comparative Results

XGBoost [10] (`max_depth=6`, `lr=0.1`) is compared against a 1D-CNN, a BiLSTM-RNN, and a Transformer encoder, all trained on the same 165-dimensional feature representation (163-dim multi-hot `DataClasses` vector, log-scaled breach volume, verification flag).

TABLE III
COMPARATIVE BENCHMARK ON TEST SET (N=157)

Model	Accuracy	Macro F1	Weighted F1
Naive baseline (predict MED)	47.1%	16.0%	30.2%
BiLSTM-RNN	44.6%	34.0%	46.8%
Transformer (2L/4H)	48.4%	42.9%	50.8%
1D-CNN (k=3,4,5)	66.2%	57.2%	65.7%
XGBoost	89.2%	83.2%	89.4%

The gap is consistent with Grinsztajn et al.’s finding that tree-based models retain an advantage on medium-sized tabular data [14]: BreachShield’s `DataClasses` vocabulary is a sparse, unordered categorical space, for which recurrent and attention architectures impose an artificial sequential or positional bias that tree splits do not.

D. Production Deployment Status

Despite its strong benchmark performance, the checkpoint-loaded XGBoost severity model is *not* the production search-path scorer. Production risk scoring is computed by a deterministic, explainable rule-based engine (`riskEngine.js`) evaluating credential-category sensitivity, source reliability, and breach recency – all confirmed, on direct source inspection, to be genuinely computed from record data rather than fixed constants. XGBoost is retained as a validated research artifact because live OSINT feeds do not reliably supply the complete `DataClasses`, volume, and verification fields the model’s feature contract requires. Separately, a live pretrained URL-transformer classifier (UrlBERT) is deployed on the actual search path and confirmed functioning against a known-phishing test URL with 0.9999 confidence.

VI. BLOCKCHAIN AUDIT TRAIL

Audit events are canonicalised (sorted-key JSON) and hashed into Merkle leaves, batched by a 16-record or 60-second window, and anchored to `AnchorRegistry.sol` on a local Hardhat EVM node.

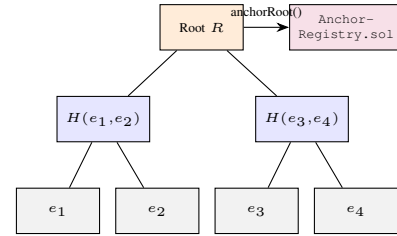


Fig. 2. Merkle batching and EVM anchoring workflow (illustrative, 4-leaf batch; production batch size is 16).

A. Current Verified Status

Direct testing of a clean local deploy confirms `AnchorRegistry.sol` **compiles successfully**. Auto-deployment on node restart currently **fails**: with `ANCHOR_PRIVATE_KEY` unset in the local environment, Hardhat’s configured account list is empty, `getSigners()` returns no deployer, and the deploy script throws at the point it attempts to read the (undefined) deployer’s address. This is a configuration defect, not a design flaw in the batching or anchoring logic itself, and is corrected by supplying the missing credential; we report it as currently unresolved rather than claim a verified deploy that does not presently succeed.

VII. VERIFICATION METHODOLOGY

Because significant portions of BreachShield were built with AI coding-agent assistance, the team adopted a policy that no capability claim would be retained in project reporting without independent, live re-verification, executed in three stages.

Stage 1 – Dual independent audit. Two separately-prompted AI audit agents, without access to each other’s output, each performed an eight-section review of the codebase

(architecture, data flow, ML pipeline, blockchain, security, testing, documentation accuracy, data-source honesty).

Stage 2 – Live cross-checking. Disagreements between the two audits were resolved by running direct, live commands against the running system – `curl` calls to live endpoints, direct source reads, and repeated inference calls to test determinism – rather than by adjudicating between the two reports.

Stage 3 – Independent third-party reproduction. A third party, without access to either prior audit’s conclusions, re-derived results from raw data and raw source in an isolated environment.

This process both confirmed genuine, previously-unverified positive results (Section VIII) and surfaced specific fabricated or broken claims, including: latency figures (round-trip search latency, client-hash timing) that existed only as hardcoded strings inside paper-generation scripts and corresponded to no measurement anywhere in the codebase; and a set of literature citations, discussed in Section II’s predecessor drafts, that were never independently checked before being written into early project documentation. We report this not to discredit the project but because verifying and correcting such claims before publication is, in our view, the more defensible response than omitting the finding.

VIII. RESULTS

TABLE IV
VERIFICATION SUMMARY

Claim	Verified status
k-anonymity search Severity classifier	Live-tested; correct prefix/bucket round trip 89.2% test accuracy; checkpoint-loaded, deterministic
Backend test suite	101/101 passing, 26 suites (both default and <code>SKIP_OTP=false</code>)
Frontend test suite	50/50 passing, 3 suites
Risk-scoring engine	Confirmed genuinely computed (source-inspected)
Live URL classifier	Confirmed functioning; 0.9999 confidence on test case
Merkle audit batching	Construction logic verified; contract compiles
Blockchain auto-deploy	Currently fails – missing deployer credential
System latency benchmarks	No measurement exists ; prior figures were fabricated

IX. DISCUSSION

The pattern underlying Table IV’s unresolved items is consistent: each was a claim that satisfied the surface shape of a requirement (an endpoint exists, a number is quoted, a deploy step runs) without satisfying its substance, and none were caught by the project’s own automated test suite. This is consistent with broader findings on LLM-assisted code and text generation producing plausible but unverified output. For a platform whose core value proposition is trustworthy breach reporting, this has a direct implication: an unverified claim about the platform’s own behaviour is not a cosmetic defect but undermines the specific trust the platform asks users to place in it.

X. LIMITATIONS AND THREATS TO VALIDITY

The k-anonymity search protocol uses the standard HIBP-style prefix-bucket construction without the stronger private-set-intersection guarantees Li et al. propose [9]; combined with OTP-gated access, the server can associate a query session with an authenticated identity even though it cannot determine the specific target within a returned bucket – a deliberate accountability/privacy trade-off, not an oversight, but one that should not be understated. The CRITICAL severity class is represented by only 9 test-set records; per-class precision/recall on this tier carries correspondingly wide uncertainty, and the class-imbalance mitigation demonstrated by Alsubaei et al. [3] (83%→98% accuracy via SMOTE) is identified here as unapplied future work rather than a completed step. Blockchain anchoring currently targets a local EVM node rather than a public chain, and auto-deployment is presently blocked by a missing credential (Section VI); public verifiability claims are therefore not made. No system-level latency benchmarks exist at the time of writing; per-request inference timings are available (UrIBERT approx. 14–31 ms) but should not be read as aggregate or percentile system performance claims.

XI. FUTURE WORK

Immediate priorities are: supplying the missing deployment credential and re-verifying a clean blockchain auto-deploy; applying SMOTE or an equivalent resampling strategy to the CRITICAL severity tier following Alsubaei et al.’s demonstrated approach [3]; and running a genuine, reproducible latency benchmark suite before any performance claim is made in future reporting. Medium-term work includes evaluating Li et al.’s private-set-intersection protocols [9] as a replacement for the current OTP-gated prefix design, and assessing public-chain anchoring following the batching pattern of Arabnouri et al. [7] once local verification is complete.

XII. CONCLUSION

BreachShield demonstrates a privacy-preserving breach intelligence architecture combining a k-anonymity search protocol, a benchmarked and reproducible ML severity classifier, and a Merkle-batched audit log, developed and verified by a final-year undergraduate team. Equally, this paper documents that AI-assisted development of the same codebase introduced specific, fabricated or non-functional claims – concerning an unrelated model population, blockchain deployment status, and unmeasured performance figures – that a three-stage, live-verification methodology was able to identify and correct prior to publication. We offer that methodology, alongside the verified system itself, as this paper’s contribution.

ACKNOWLEDGMENT

The authors thank Prof. Sushma T. M. for guidance throughout Project Phase II (BIS786).

REFERENCES

- [1] A. Aljofey, Q. Jiang, Q. Qu, M. Huang, and J.-P. Niyigena, "An Effective Phishing Detection Model Based on Character Level Convolutional Neural Network from URL," *Electronics*, vol. 9, no. 9, p. 1514, 2020.
- [2] O. K. Sahingoz, E. Buber, and E. Kuğu, "DEPHIDES: Deep Learning Based Phishing Detection System," *IEEE Access*, vol. 12, pp. 8052–8070, 2024.
- [3] F. S. Alsubaei, A. A. Almazroi, and N. Ayub, "Enhancing Phishing Detection: A Novel Hybrid Deep Learning Framework for Cybercrime Forensics," *IEEE Access*, vol. 12, pp. 8373–8389, 2024.
- [4] B. V. Pavani, D. Mahitha, and B. Uma Maheswari, "Enhancing Online Safety: Phishing URL Detection Using Machine Learning and Explainable AI," in *Proc. 2024 15th ICCCNT*, IEEE, 2024.
- [5] M. Tawfik, A. A. Abu-Ein, A. H. Abdelhaliem, Y. M. Al-Sharo, and I. S. Fathi, "Explainable Few-Shot Learning with Modern BERT for Detecting Emerging Phishing Attacks Using XF-PhishBERT," *Scientific Reports*, vol. 15, art. 42821, 2025.
- [6] P. Kühn, K. Wittorf, and C. Reuter, "Navigating the Shadows: Manual and Semi-Automated Evaluation of the Dark Web for Cyber Threat Intelligence," *IEEE Access*, 2024.
- [7] A. Arabnouri, S. Eissazadeh, and A. Shafieinejad, "A Secure Auditable Log based on Blockchain," *Monadi*, vol. 13, no. 2, pp. 75–86, 2024.
- [8] L. Sweeney, "k-Anonymity: A Model for Protecting Privacy," *Int. J. Uncertainty, Fuzziness Knowl.-Based Syst.*, vol. 10, no. 5, pp. 557–570, 2002.
- [9] L. Li, B. Pal, J. Ali, N. Sullivan, R. Chatterjee, and T. Ristenpart, "Protocols for Checking Compromised Credentials," in *Proc. ACM CCS*, 2019, pp. 1387–1403.
- [10] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
- [11] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Advances in Cryptology — CRYPTO '87*, Springer, 1988, pp. 369–378.
- [12] S. A. Crosby and D. S. Wallach, "Efficient Data Structures for Tamper-Evident Logging," in *Proc. USENIX Security Symp.*, 2009.
- [13] K. El Emam and F. K. Dankar, "Protecting Privacy Using k-Anonymity," *J. Amer. Med. Inform. Assoc.*, vol. 15, no. 5, pp. 627–637, 2008.
- [14] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?," in *Proc. NeurIPS*, 2022.
- [15] P. Grassi, M. Garcia, and J. Fenton, "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST SP 800-63B, 2020.
- [16] RFC 7519, "JSON Web Token (JWT)," IETF, 2015.